WeSee AI Automation Platform

Table of Contents

System Overview

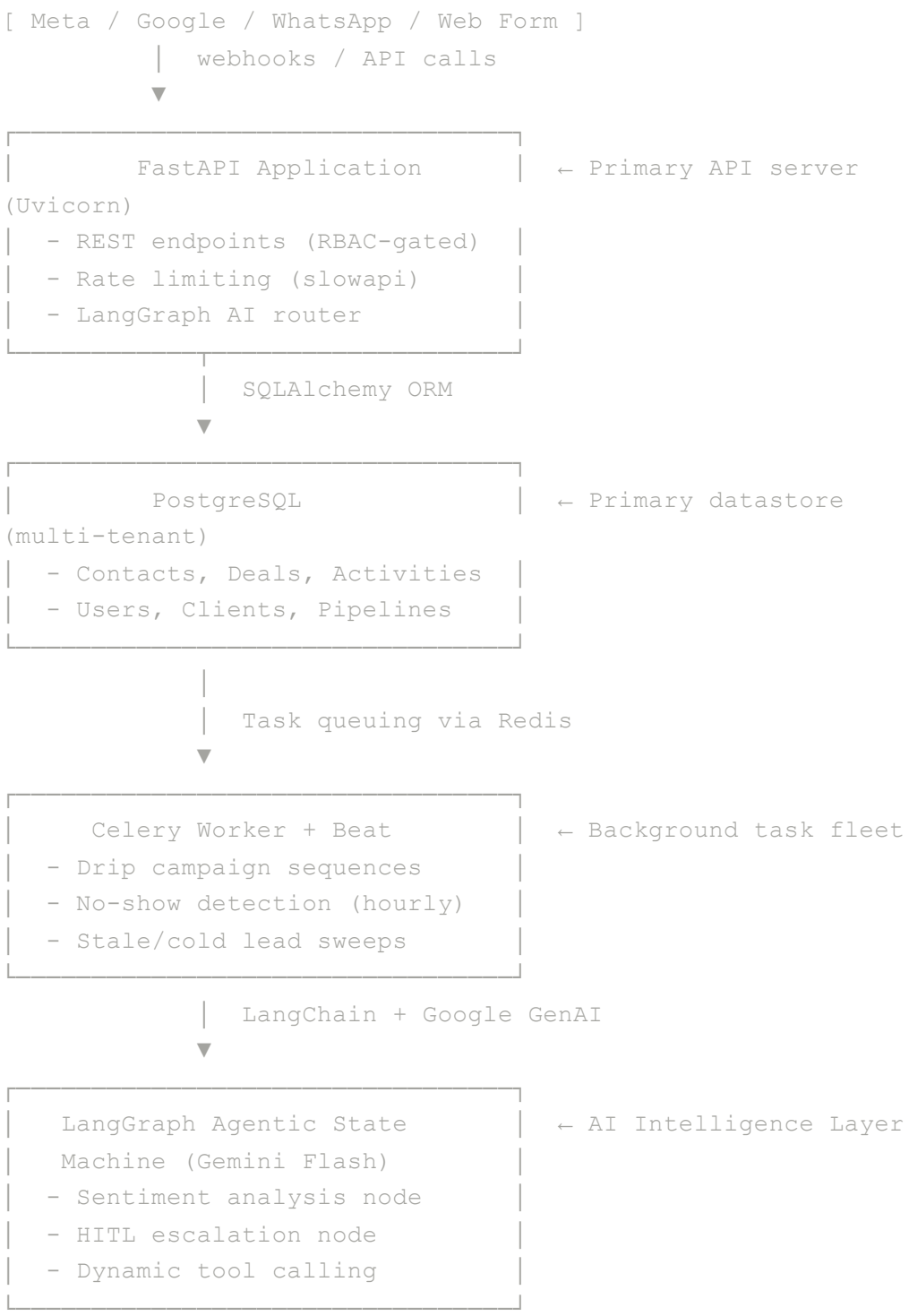
WeSee is a production-ready AI Automation Platform designed to eliminate manual friction in B2B and B2C sales pipelines. It operates as a multi-tenant backend, meaning a single deployment serves multiple isolated company accounts simultaneously.

The platform has three primary intelligence layers:

Layer	Responsibility
Ingestion Engine	Captures leads from Meta Ads, Google Ads, WhatsApp, and web forms in real time
AI Agent (LangGraph)	Conducts reasoning-aware sales conversations, checks calendars, and autonomously books appointments
Celery Worker Fleet	Runs background tasks — drip campaigns, lead scoring penalties, stale re-engagement, and no-show follow-ups

---

Architecture & Tech Stack



Technology Versions

Technology	Role
FastAPI	Async REST API framework

Technology	Role
PostgreSQL	Primary relational database
SQLAlchemy 2.0	ORM with row-level locking
Alembic	Database schema migrations
Redis	Celery broker, backend, and round-robin rep queue
Celery 5 + **Beat**	Distributed task queue and periodic scheduler
LangGraph	Stateful AI agent graph execution
LangChain Google GenAI	Gemini Flash LLM integration
python-jose	JWT creation and validation
itsdangerous	Cryptographically signed password reset tokens
bcrypt	Password hashing
slowapi	Rate limiting (leaky-bucket per route)
SendGrid	Transactional email dispatch
Google Calendar API	Real-time slot fetching and event creation

---

Project Structure

```
PythonAutomation/
├── main.py                    # FastAPI app: webhooks, CRM
endpoints, RBAC routes
├── auth.py                    # Password hashing, JWT token
creation
├── config.py                  # Pydantic Settings (validates env
vars at boot)
├── database.py                # SQLAlchemy engine, session
factory, context manager
├── dependencies.py            # get_current_user, require_admin
FastAPI dependencies
├── models.py                  # All SQLAlchemy ORM models
(Client, User, Contact, ...)
├── schemas.py                 # Pydantic request/response
schemas
├── services.py                # Lead scoring (with row-locking),
round-robin distrib.
├── communications.py          # Email (SendGrid) and WhatsApp
(Meta API) dispatch
├── celery_app.py              # Celery + Beat configuration and
scheduled task registry
├── celery_worker.py           # Campaign executor, cold/stale
lead sweeps, bulk email
├── tasks.py                   # No-show detector, AI follow-up
summary generator
├── workflows.py               # Campaign step definitions
├── routers/
│   ├── auth.py                # /api/auth/* — login, signup,
/me, password reset
│   ├── admin.py               # /api/admin/* — internal user
management
│   ├── ai_agent.py            # /api/ai/* — AI chat and content
generation
│   ├── google_auth.py         # Google OAuth2 flow
│   └── meetings.py            # Calendar integration and
rescheduling
├── utils/
│   ├── agent_graph.py         # LangGraph graph definition
(nodes, edges, routing)
│   ├── ai_engine.py           # High-level AI engine interface
│   ├── ai_tools.py            # LangChain @tools:
check_calendar_availability, book_appointment, get_lead_score
│   ├── calendar.py            # Google Calendar slot generation
logic
```

```
│   ├── logger.py            # Structured JSON logger
(timezone-aware UTC timestamps)
│   ├── prompts.py           # Centralized system voice and
prompt templates
│   ├── security.py          # Reschedule/unsubscribe token
generation
│   └── security_limiter.py  # slowapi limiter instance
├── alembic/                 # Database migration scripts
├── tests/                   # Full pytest suite (7 IAM +
functional tests)
└── .env.example             # Environment variable template
```

Security & Identity (IAM)

Multi-Tenant Isolation

Every resource in the database is scoped to a `client_id`. This is the foundational tenant boundary.

- **At the API layer**: Every authenticated request uses `get_current_user` to retrieve the user's `client_id` from their JWT. All database queries are hard-filtered by this value — users are physically incapable of reading another tenant's data.
- **At the admin layer**: When an admin creates a new user via `POST /api/admin/users`, the system **automatically injects** the requester's `client_id` — the payload is never trusted to supply it. This prevents cross-tenant account creation.
- **At the signup layer**: `POST /api/auth/signup` creates a `Client` record and an admin `User` in a **single atomic transaction** with `db.flush()`. If user creation fails (e.g., duplicate email), the entire transaction rolls back — no orphaned `Client` records are left behind.

JWT Role-Based Access Control (RBAC)

Upon login, the JWT encodes three claims:

```
{
  "sub": "user_id",
  "client_id": 42,
  "role": "admin"
}
```

Two dependency tiers control access:

Dependency	Enforces
`get_current_user`	Valid JWT + active user account
`require_admin`	`get_current_user` + `role == "admin"` (returns HTTP 403 otherwise)

Role visibility rules (RBAC + RLS combined):

Role	Contact Visibility
`admin`	All contacts within their `client_id`
`sales_rep`	Only contacts where `owner_id == user.id`

Password Reset Security

1. `POST /api/auth/forgot-password` uses `itsdangerous.URLSafeTimedSerializer` to generate a **cryptographically signed, time-limited token** (1 hour TTL). No token is stored in the database — the signature is the proof of authenticity.
2. Non-existent email addresses return an **identical success response** to prevent user enumeration attacks via the API.
3. `POST /api/auth/reset-password` validates the token's signature and expiry before updating the hash.

Rate Limiting

All sensitive routes are protected by `slowapi` (leaky bucket):

Route	Limit
`POST /api/auth/login`	5 / minute
`POST /api/auth/signup`	5 / minute
`POST /api/webhooks/meta`	100 / minute
`POST /api/webhooks/google`	100 / minute
`POST /api/webhooks/whatsapp`	100 / minute
`POST /api/webhooks/incoming_msg`	100 / minute
`POST /api/ai/chat`	20 / minute

Exceeding a limit returns `HTTP 429 Too Many Requests`.

Core Modules

Ingestion Engine

Files: `main.py`, `communications.py`, `utils/__init__.py`

The platform ingests leads from four sources, all routed through a shared deduplication gate:

```
Inbound Lead
     │
     ▼
handle_duplicate_prevention(db, email, phone, client_id)
     │
     ├── Duplicate found? → Update last_active timestamp → Return
     │
     └── New lead → Create Contact → Assign to next sales rep (round-robin via Redis)
                                   → Fire Celery task: execute_workflow_step()
```

Channels:

Endpoint	Source	Key Logic
`POST /api/webhooks/meta`	Meta Ads	Dedup by email + phone
`POST /api/webhooks/google`	Google Ads	Fuzzy field extraction from `user_column_data`
`POST /api/webhooks/whatsapp`	WhatsApp Cloud API	Dedup by E.164 phone number
`POST /api/webhooks/incoming_msg`	Inbound WhatsApp reply	Triggers kill switch (stops automation), fires lead score `+40` for `replied` event
`POST /api/leads/web`	Web forms / QR codes	Priority: `source` field overrides `utm_source`

Outbound tracking (injected into every email by `send_email_outbound`):

- **Open pixel**: Transparent 1×1 GIF at `/api/track/open/{contact_id}` (+10 lead score)
- **Click tracking**: All `href` links wrapped with signed tokens at `/api/track/click/{token}` (+20 lead score)
- **Unsubscribe**: Signed one-click unsubscribe link that sets `is_subscribed = False` (kill switch)

AI Intelligence Layer

Files: `utils/agent_graph.py`, `utils/ai_tools.py`, `utils/ai_engine.py`, `routers/ai_agent.py`

The AI agent is a **LangGraph stateful graph** powered by `gemini-3-flash-preview`. Each conversation runs through a deterministic pipeline of nodes:

```
User Message
     |
     ▼
[sentiment_analyzer] ─── Frustrated / wants human? ──▶ [escalation_node] ──▶ END
     | (Neutral / Positive)
     ▼
[context_retriever]  ← Fetches last 10 CRM activities for this lead
     |
     ▼
[call_model]         ← Gemini Flash with system voice, context, lead score, tools bound
     |
     ├── No tool calls? ──────────────────────────────────────────────────▶ END
     |
     └── Tool calls? ──▶ [custom_tool_node] ──▶ (result injected) ──▶ [call_model] ──▶ ...
```

Available Tools (dynamic function calling):

Tool	Description
`check_calendar_availability(date_str, user_id)`	Queries Google Calendar for free 30-min slots on a given date
`book_appointment(user_id, contact_email, contact_name, date_str, time_str, client_id, owner_id)`	Creates a Google Calendar event, generates a reschedule token, and logs the activity

Tool	Description
`get_lead_score(contact_email)`	Returns the current lead score and temperature label (Hot / Warm / Cold)

State Shape (`AgentState`):

```
{
    "messages":          list[BaseMessage],   # Full conve
    "user_id":           int,                 # The sales
    "contact_id":        int,                 # The lead's
    "client_id":         int,                 # The tenant
    "owner_id":          int,                 # For bookin
    "sentiment":         str,                 # Output of
    "context_snapshot":  str,                 # Last 10 CR
    "escalation_flag":   bool                 # Triggers H
}
```

Content Generation: `POST /api/ai/generate-content` uses the same LLM but in a direct one-shot invocation (not a graph) to produce platform-tailored marketing posts from CRM history context.

Autonomous Background Sweeps

Files: `celery_app.py` , `celery_worker.py` , `tasks.py`

The Celery Beat scheduler runs three autonomous sweeps on a fixed schedule (timezone: `Asia/Kolkata` , internal clock: UTC):

Campaign Executor: `execute_workflow_step`

The universal campaign interpreter. It reads the step configuration, dispatches the correct channel (email, WhatsApp template, WhatsApp text), logs the CRM activity, and schedules the next step with exponential backoff retry (up to 4 attempts: 60s, 120s, 240s, 480s).

Compliance checks run before every step:

1. `is_subscribed == False` → Immediately abort (unsubscribe kill switch)
2. `status in ['replied', 'booked']` → Immediately abort (engagement kill switch)

Scheduled Tasks (Celery Beat)

Schedule	Task	Logic
Top of every hour	`check for no shows`	Finds `meeting_booked` activities where the start time was 30+ minutes ago with no `meeting_completed` log. Sends a reschedule link email and logs `no_show_followup_sent` to prevent re-triggering.
Daily at 18:00 IST	`sweep cold leads`	Finds leads with `status='new'` created 30+ days ago. Drops them into the `default_web_nurture` drip campaign and sets `status='nurturing'`.
Daily at 18:15 IST	`sweep stale leads`	Finds leads with `last_active` 30+ days ago. Triggers `breakup_reengagement` campaign, applies a `-20` lead score penalty (with automatic Cold/Warm/Hot threshold logging), and resets the `last_active` clock.

Lead Scoring Engine (`services.py`)

Uses `SELECT ... FOR UPDATE` (row-level locking) to prevent race conditions from concurrent Celery task writes.

Event	Score Delta

Event	Score Delta
`email_opened`	+10
`link_clicked`	+20
`replied`	+40
`booked`	+60
`stale`	-20
`no_show`	-50
`email_bounce`	-30
`unsubscribed`	-100

When a score crosses a Hot/Warm/Cold threshold, a `threshold_transition` activity is automatically logged in the CRM timeline.

API Reference Summary

Authentication (`/api/auth`)

Method	Path	Auth	Description
`POST`	`/api/auth/signup`	Public	Create company + admin account (atomic transaction)
`POST`	`/api/auth/login`	Public	OAuth2 password flow, returns JWT
`GET`	`/api/auth/me`	Bearer	Returns current user profile
`POST`	`/api/auth/forgot-password`	Public	Dispatch signed reset link via email
`POST`	`/api/auth/reset-password`	Public	Validate token, update password

Admin Management (`/api/admin`)

Method	Path	Auth	Description
`POST`	`/api/admin/users`	Admin JWT	Create a user in the admin's tenant
`GET`	`/api/admin/users`	Admin JWT	List all users in the admin's tenant

CRM & Leads (`/api`)

Method	Path	Auth	Description
`GET`	`/api/contacts`	Bearer	Fetch contacts (RLS + RBAC applied)
`POST`	`/api/leads/segment`	Bearer	Dynamic filter-based contact segmentation
`POST`	`/api/leads/bulk-email`	Bearer	Compliance-filtered bulk email fan-out
`GET`	`/api/contacts/{id}/timeline`	Bearer	Chronological CRM activity log
`GET`	`/api/pipelines/{id}/kanban`	Bearer	Kanban deal board
`GET`	`/api/reports/attribution`	Bearer	Campaign ROI (UTM source → deal count)

AI Agent (`/api/ai`)

Method	Path	Auth	Description
`POST`	`/api/ai/chat`	Public*	LangGraph sales agent conversation
`POST`	`/api/ai/generate-content`	Public*	AI marketing content generation

> *Consider adding Bearer token protection to AI routes in production.

Local Setup & Testing

Prerequisites

- Python 3.11+
- PostgreSQL
- Redis

1. Clone & Create Virtual Environment

```
git clone <repo>
cd PythonAutomation
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

2. Configure Environment Variables

```
cp .env.example .env
```

Open `.env` and fill in the required values:

```
# Database (required)
DATABASE_URL=postgresql://user:password@127.0.0.1:5432/we

# Redis (required for Celery)
REDIS_URL=redis://localhost:6379/0

# JWT Security (change in production!)
SECRET_KEY=your-super-secret-key-min-32-chars
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=1440

# Meta Webhooks
META_VERIFY_TOKEN=your_meta_verify_token
META_ACCESS_TOKEN=your_meta_access_token
META_PHONE_NUMBER_ID=your_phone_number_id

# Google Cloud
GOOGLE_API_KEY=your_gemini_api_key
GOOGLE_CLIENT_ID=your_oauth_client_id
GOOGLE_CLIENT_SECRET=your_oauth_client_secret
GOOGLE_REDIRECT_URI=http://localhost:8000/api/auth/google

# SendGrid (leave blank in dev — emails are simulated wit
SENDGRID_API_KEY=

# App Environment
ENVIRONMENT=development
BASE_URL=http://localhost:8000
```

> **Security Note**: If `ENVIRONMENT=production`, the server will refuse to start with the default `SECRET_KEY`. This is enforced in `config.py` via a Pydantic `@model_validator`.

3. Apply Database Migrations

```
alembic upgrade head
```

4. Seed the Initial Admin User

```
python seed_user.py
# Output: ✅ Success! Created Admin -> Email: admin@wesee
```

5. Start the FastAPI Server

```
uvicorn main:app --reload
```

The interactive API docs are available at: **http://localhost:8000/docs**

6. Start the Celery Worker

In a second terminal:

```
source venv/bin/activate
celery -A celery_app.celery_app worker --loglevel=info
```

7. Start the Celery Beat Scheduler

In a third terminal:

```
source venv/bin/activate
celery -A celery_app.celery_app beat --loglevel=info
```

8. Run the Test Suite

```
./venv/bin/pytest tests/test_iam.py -v
```

Expected output:

```
tests/test_iam.py::test_company_signup_success          PASSED
[ 14%]
tests/test_iam.py::test_company_signup_duplicate_email  PASSED
[ 28%]
tests/test_iam.py::test_admin_creates_user_securely     PASSED
[ 42%]
tests/test_iam.py::test_rbac_sales_rep_cannot_create_users
PASSED [ 57%]
tests/test_iam.py::test_password_reset_flow             PASSED
[ 71%]
tests/test_iam.py::test_login_success                   PASSED
[ 85%]
tests/test_iam.py::test_get_current_user_details        PASSED
[100%]

======================== 7 passed in 3.6s
========================
```

To run the full suite:

```
./venv/bin/pytest tests/ -v
```

Design Decisions & Notes

- **Stateless Password Resets**: Using `itsdangerous` signed tokens avoids adding a `password_reset_tokens` table to the database, keeping the schema lean and the auth system stateless.
- **Row-Level Locking for Scoring**: `SELECT ... FOR UPDATE` in `update_lead_score` prevents duplicate score writes when multiple Celery workers process the same lead concurrently.

- **Round-Robin via Redis LPOP/RPUSH**: The sales rep assignment queue is a Redis list. This gives O(1) assignment and automatic skip-over of inactive/deleted reps without any database polling.
- **Timezone Standard**: All datetime values are `timezone.utc` -aware (`datetime.now(timezone.utc)`). SQLAlchemy column defaults use `lambda: datetime.now(timezone.utc)` (not `datetime.utcnow`, which is deprecated in Python 3.12+).
- **LangGraph Tool Injection**: The `client_id` and `owner_id` are injected into tool arguments by the `custom_tool_node` — they are never passed through the LLM prompt to prevent prompt injection from overriding tenant context.